



IPv6双协议栈实验报告

实验目标

实现一个支持IPv4和IPv6双协议栈的网络层协议处理模块，包括IPv6数据包的收发处理、IPv4-IPv6地址映射机制、以及双协议栈的互操作性。

1. 系统架构设计

1.1 总体架构

本实验实现的IPv6双协议栈采用分层架构设计：

```
应用层 (UDP/TCP)
  ↓
网络层 (IPv4/IPv6)
  ↓
数据链路层 (Ethernet/ARP)
  ↓
物理层 (Driver)
```

1.2 协议栈特性

- 双协议栈支持：同时支持IPv4和IPv6协议
- 地址映射机制：实现IPv4到IPv6的地址映射(::FFFF:x.x.x.x)
- 统一接口：为上层协议提供统一的网络服务接口
- 互操作性：IPv4和IPv6协议之间可以通过映射地址实现互通

2. 数据结构设计

2.1 IPv6头部结构

```
#pragma pack(1)
typedef struct ip6_hdr {
    uint32_t version_tc_flowlabel;    // 版本(4位),流量类别(8位),流标签(20位)
    uint16_t payload_len;              // 有效载荷长度
    uint8_t next_header;              // 下一个头部
    uint8_t hop_limit;                // 跳数限制
    uint8_t src_ip[NET_IP6_LEN];      // 源IPv6地址
    uint8_t dst_ip[NET_IP6_LEN];      // 目标IPv6地址
} ip6_hdr_t;
#pragma pack()
```

设计要点:

- `version_tc_flowlabel`: 32位字段, 包含版本号(4位)、流量类别(8位)和流标签(20位)
- `payload_len`: 16位字段, 表示有效载荷长度(不包括IPv6头部)
- `next_header`: 8位字段, 指示下一个头部的协议类型
- `hop_limit`: 8位字段, 类似IPv4的TTL字段
- 源和目标地址各占128位(16字节)

2.2 IPv4映射IPv6地址格式

```
// IPv4映射的IPv6地址前缀 (::FFFF:0:0/96)
const uint8_t IPV4_MAPPED_PREFIX[NET_IP6_LEN] = {
    0x00, 0x00, 0x00, 0x00, // 10字节的0
    0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0xff, 0xff, // 0:0:0:0:0:0:FFFF:
    0x00, 0x00, 0x00, 0x00 // IPv4地址位置
};
```

映射机制:

- 前80位(10字节)全为0
- 接下来16位为0xFFFF

- 最后32位存储IPv4地址
- 格式: `::FFFF:192.168.1.100`

2.3 网络接口配置

```
extern uint8_t net_if_ip[NET_IP_LEN];    // IPv4地址
extern uint8_t net_if_ip6[16];           // IPv6地址
```

3. 关键函数实现

3.1 IPv6数据包接收处理

```
void ip6_in(buf_t *buf, uint8_t *src_mac)
```

算法流程:

1. 长度验证

```
if (buf->len < sizeof(ip6_hdr_t)) {
    return; // 数据包太小, 丢弃
}
```

2. 版本号检查

```
ip6_hdr_t *ip6_hdr = (ip6_hdr_t *)buf->data;
uint8_t version = (ntohl(ip6_hdr->version_tc_flowlabel) >> 28) & 0xF;
if (version != IP_VERSION_6) {
    return; // 不是IPv6数据包
}
```

3. 载荷长度验证

```
uint16_t payload_len = ntohs(ip6_hdr->payload_len);
if (payload_len + sizeof(ip6_hdr_t) > buf->len) {
    return; // 载荷长度不匹配
}
```

4. 目标地址检查

```
if (memcmp(ip6_hdr->dst_ip, net_if_ip6, NET_IP6_LEN) != 0) {
    // 检查是否是IPv4映射地址
    uint8_t ipv4_addr[NET_IP_LEN];
    if (is_ip4_mapped_ip6(ip6_hdr->dst_ip) &&
        ip6_to_ip4_addr(ip6_hdr->dst_ip, ipv4_addr) &&
        memcmp(ipv4_addr, net_if_ip, NET_IP_LEN) == 0) {
        // 是发给本机IPv4的映射地址，继续处理
    } else {
        return; // 不是发给本机的数据包
    }
}
```

5. 头部移除和上层协议处理

```
buf_remove_header(buf, sizeof(ip6_hdr_t));
uint8_t next_header = ip6_hdr->next_header;
uint8_t src_ip6[NET_IP6_LEN];
memcpy(src_ip6, ip6_hdr->src_ip, NET_IP6_LEN);

if (net_in6(buf, next_header, src_ip6) < 0) {
    // 上层协议无法处理
    return;
}
```

3.2 IPv6数据包发送处理

```
void ip6_out(buf_t *buf, uint8_t *ip6, net_protocol_t protocol)
```

算法流程：

1. MTU检查

```
int max_payload_len = ETHERNET_MAX_TRANSPORT_UNIT - sizeof(ip6_hdr_t);
if (buf->len > max_payload_len) {
    buf->len = max_payload_len; // 截断超大数据包
}
```

2. 添加IPv6头部

```
uint16_t payload_len = buf->len;
buf_add_header(buf, sizeof(ip6_hdr_t));
ip6_hdr_t *ip6_hdr = (ip6_hdr_t *)buf->data;
```

3. 设置头部字段

```
// 版本号设为6，流量类别和流标签设为0
uint32_t version_tc_flowlabel = IP_VERSION_6 << 28;
ip6_hdr->version_tc_flowlabel = htonl(version_tc_flowlabel);

ip6_hdr->payload_len = htons(payload_len);
ip6_hdr->next_header = protocol;
ip6_hdr->hop_limit = IP6_DEFAULT_HOP_LIMIT;

memcpy(ip6_hdr->src_ip, net_if_ip6, NET_IP6_LEN);
memcpy(ip6_hdr->dst_ip, ip6, NET_IP6_LEN);
```

4. 地址解析和发送

```
uint8_t ipv4_addr[NET_IP_LEN];
if (is_ip4_mapped_ip6(ip6) && ip6_to_ip4_addr(ip6, ipv4_addr)) {
    // IPv4映射地址，使用ARP协议
    arp_out(buf, ipv4_addr);
} else {
    // 纯IPv6地址，使用广播MAC地址(简化处理)
    uint8_t broadcast_mac[NET_MAC_LEN] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
    ethernet_out(buf, broadcast_mac, NET_PROTOCOL_IP6);
}
```

3.3 地址转换函数

3.3.1 IPv4到IPv6映射地址转换

```
int ip4_to_ip6_addr(const uint8_t *ip4_addr, uint8_t *ip6_addr)
```

功能：将IPv4地址转换为IPv4映射的IPv6地址

算法：

1. 复制IPv4映射前缀(80位0 + 16位0xFFFF)
2. 将IPv4地址复制到最后32位

```
memcpy(ip6_addr, IPV4_MAPPED_PREFIX, NET_IP6_LEN);
memcpy(ip6_addr + 12, ip4_addr, NET_IP_LEN);
```

3.3.2 IPv4映射地址检测

```
int is_ip4_mapped_ip6(const uint8_t *ip6_addr)
```

功能：检测IPv6地址是否为IPv4映射地址

算法：

1. 检查前80位是否全为0
2. 检查第81-96位是否为0xFFFF

```
// 检查前10个字节是否为0
for (int i = 0; i < 10; i++) {
    if (ip6_addr[i] != 0) {
        return 0;
    }
}

// 检查第11、12字节是否为0xFF
if (ip6_addr[10] != 0xFF || ip6_addr[11] != 0xFF) {
    return 0;
}

return 1; // 是IPv4映射地址
```

3.3.3 IPv6映射地址到IPv4转换

```
int ip6_to_ip4_addr(const uint8_t *ip6_addr, uint8_t *ip4_addr)
```

功能：从IPv4映射的IPv6地址中提取IPv4地址

算法：

1. 首先验证是否为IPv4映射地址
2. 提取最后32位作为IPv4地址

```
if (!is_ip4_mapped_ip6(ip6_addr)) {  
    return 0; // 不是IPv4映射地址  
}  
  
memcpy(ip4_addr, ip6_addr + 12, NET_IP_LEN);  
return 1;
```

4. 测试框架设计

4.1 测试架构

测试文件 `ip6_test.c` 采用单元测试框架，包含8个主要测试模块：

- IPv6地址转换测试
- IPv6头部解析测试
- IPv6数据包发送测试
- IPv6和IPv4映射地址互操作性测试
- IPv6数据包输入处理测试
- 大数据包处理测试
- 错误处理测试
- IPv6双协议栈集成测试

```

struct {
    char *test_name;
    void (*test_func)(void);
} test_cases[] = {
    {"IPv6 address conversion", test_ipv6_address_conversion},
    {"IPv6 header parsing", test_ipv6_header_parsing},
    {"IPv6 packet output", test_ipv6_packet_output},
    {"IPv6-IPv4 interoperability", test_ipv6_ipv4_interoperability},
    {"IPv6 packet input", test_ipv6_packet_input},
    {"Large packet handling", test_ipv6_large_packet_handling},
    {"Error handling", test_ipv6_error_handling},
    {"Dual stack integration", test_ipv6_dual_stack_integration}
};

```

4.2 测试辅助函数

4.2.1 断言函数

```

void test_assert(int condition, const char *test_name) {
    if (condition) {
        printf("✓ PASS: %s\n", test_name);
        test_passed++;
    } else {
        printf("X FAIL: %s\n", test_name);
        test_failed++;
    }
}

```

4.2.2 IPv6地址打印函数

```

void print_ipv6_addr(const uint8_t *ip6) {
    printf("%02x%02x:%02x%02x:%02x%02x:%02x%02x:%02x%02x:%02x%02x",
        ip6[0], ip6[1], ip6[2], ip6[3], ip6[4], ip6[5], ip6[6], ip6[7],
        ip6[8], ip6[9], ip6[10], ip6[11], ip6[12], ip6[13], ip6[14], ip6[15]);
}

```


4.3 详细测试设计

4.3.1 地址转换测试

```
void test_ipv6_address_conversion()
```

测试目标：验证IPv4-IPv6地址转换函数的正确性

测试步骤：

1. IPv4到IPv6映射转换测试

```
uint8_t ipv4_addr[] = {192, 168, 1, 100};  
uint8_t ipv6_mapped[NET_IP6_LEN];  
int result = ip4_to_ip6_addr(ipv4_addr, ipv6_mapped);
```

2. 映射地址格式验证

```
uint8_t expected_mapped[] = {  
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,  
    0x00, 0x00, 0xFF, 0xFF, 192, 168, 1, 100  
};  
test_assert(memcmp(ipv6_mapped, expected_mapped, NET_IP6_LEN) == 0,  
    "IPv4-mapped IPv6 address format is correct");
```

3. 映射地址检测测试

```
test_assert(is_ip4_mapped_ip6(ipv6_mapped) == 1,  
    "Should detect IPv4-mapped address");
```

4. IPv6到IPv4提取测试

```
uint8_t extracted_ipv4[NET_IP_LEN];  
result = ip6_to_ip4_addr(ipv6_mapped, extracted_ipv4);  
test_assert(memcmp(extracted_ipv4, ipv4_addr, NET_IP_LEN) == 0,  
    "Extracted IPv4 address should match original");
```

5. 纯IPv6地址测试

```
uint8_t pure_ipv6[] = {
    0x20, 0x01, 0x0d, 0xb8, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01
};
test_assert(is_ip4_mapped_ip6(pure_ipv6) == 0,
    "Pure IPv6 address should not be detected as IPv4-mapped");
```

4.3.2 IPv6头部解析测试

```
void test_ipv6_header_parsing()
```

测试目标：验证IPv6数据包头部字段的正确解析

测试数据构造：

```
ip6_hdr_t *ip6_hdr = (ip6_hdr_t *)test_buf.data;

// 设置版本号、流量类别、流标签
uint32_t version_tc_flowlabel = (IP_VERSION_6 << 28) | (0 << 20) | 0x12345;
ip6_hdr->version_tc_flowlabel = htonl(version_tc_flowlabel);
ip6_hdr->payload_len = htons(20);
ip6_hdr->next_header = NET_PROTOCOL_UDP;
ip6_hdr->hop_limit = IP6_DEFAULT_HOP_LIMIT;
```

验证测试：

```
uint8_t version = (ntohl(ip6_hdr->version_tc_flowlabel) >> 28) & 0xF;
test_assert(version == IP_VERSION_6, "IPv6 version field parsed correctly");

uint32_t flow_label = ntohl(ip6_hdr->version_tc_flowlabel) & 0xFFFFF;
test_assert(flow_label == 0x12345, "Flow label parsed correctly");
```

4.3.3 数据包发送测试

```
void test_ipv6_packet_output()
```

测试目标：验证IPv6数据包的正确发送和头部字段设置

测试数据准备：

```
// 创建测试载荷数据
char test_data[] = "Hello IPv6 World!";
buf_init(&test_buf, strlen(test_data));
memcpy(test_buf.data, test_data, strlen(test_data));

// 设定目标IPv6地址
uint8_t dst_ipv6[] = {
    0x20, 0x01, 0x0d, 0xb8, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02
};
```

发送操作测试：

```
// 记录原始数据长度
size_t original_len = test_buf.len;

// 执行IPv6数据包发送
ip6_out(&test_buf, dst_ipv6, NET_PROTOCOL_UDP);

// 验证IPv6头部是否正确添加
test_assert(test_buf.len == original_len + sizeof(ip6_hdr_t),
    "IPv6 header added correctly");
```

头部字段验证：

```

ip6_hdr_t *ip6_hdr = (ip6_hdr_t *)test_buf.data;

// 验证版本号字段
uint8_t version = (ntohl(ip6_hdr->version_tc_flowlabel) >> 28) & 0xF;
test_assert(version == IP_VERSION_6, "IPv6 version set correctly");

// 验证载荷长度字段
test_assert(ntohs(ip6_hdr->payload_len) == original_len,
            "Payload length set correctly");

// 验证下一头部字段
test_assert(ip6_hdr->next_header == NET_PROTOCOL_UDP,
            "Next header field set correctly");

// 验证跳数限制字段
test_assert(ip6_hdr->hop_limit == IP6_DEFAULT_HOP_LIMIT,
            "Hop limit set correctly");

// 验证源地址字段
test_assert(memcmp(ip6_hdr->src_ip, net_if_ip6, NET_IP6_LEN) == 0,
            "Source IPv6 address set correctly");

// 验证目标地址字段
test_assert(memcmp(ip6_hdr->dst_ip, dst_ipv6, NET_IP6_LEN) == 0,
            "Destination IPv6 address set correctly");

```

4.3.4 互操作性测试

```
void test_ipv6_ipv4_interoperability()
```

测试目标：验证IPv4和IPv6协议的互操作性，确保IPv4映射地址能够正确处理

地址转换准备：

```
// 创建标准IPv4地址
uint8_t ipv4_addr[] = {192, 168, 1, 100};

// 转换为IPv4映射的IPv6地址
uint8_t ipv6_mapped[NET_IP6_LEN];
ip4_to_ip6_addr(ipv4_addr, ipv6_mapped);
```

数据包构造测试：

```
// 创建互操作测试数据
char test_data[] = "IPv4-IPv6 interop test";
buf_init(&test_buf, strlen(test_data));
memcpy(test_buf.data, test_data, strlen(test_data));

// 记录原始长度用于后续验证
size_t original_len = test_buf.len;
```

IPv6发送IPv4映射地址测试：

```
// 使用IPv6协议发送到IPv4映射地址
ip6_out(&test_buf, ipv6_mapped, NET_PROTOCOL_UDP);

// 验证数据包结构正确性
test_assert(test_buf.len == original_len + sizeof(ip6_hdr_t),
            "IPv6 header added to IPv4-mapped address packet");
```

映射地址格式验证：

```
ip6_hdr_t *ip6_hdr = (ip6_hdr_t *)test_buf.data;

// 验证目标地址字段
test_assert(memcmp(ip6_hdr->dst_ip, ipv6_mapped, NET_IP6_LEN) == 0,
            "Destination address set to IPv4-mapped IPv6 address");

// 验证地址映射检测功能
test_assert(is_ip4_mapped_ip6(ip6_hdr->dst_ip) == 1,
            "Packet destination address correctly detected as IPv4-mapped");
```

双向转换验证：

```
// 验证能够从映射地址提取原始IPv4地址
uint8_t extracted_ipv4[NET_IP_LEN];
int result = ip6_to_ip4_addr(ip6_hdr->dst_ip, extracted_ipv4);

test_assert(result == 1,
             "IPv4 address successfully extracted from mapped address");
test_assert(memcmp(extracted_ipv4, ipv4_addr, NET_IP_LEN) == 0,
             "Extracted IPv4 address matches original address");
```

测试场景覆盖：

- IPv4地址 → IPv6映射地址 → IPv6数据包发送
- IPv6数据包接收 → 映射地址识别 → IPv4地址提取
- 混合网络环境下的协议栈兼容性验证

4.3.5 错误处理测试

```
void test_ipv6_error_handling()
```

测试目标：验证各种错误情况的正确处理

错误场景：

1. 数据包长度不足

```
buf_init(&test_buf, sizeof(ip6_hdr_t) - 1);
ip6_in(&test_buf, src_mac);
// 应该被拒绝
```

2. 错误的版本号

```
uint32_t wrong_version = (IP_VERSION_4 << 28);
ip6_hdr->version_tc_flowlabel = htonl(wrong_version);
// 应该被拒绝
```

3. 载荷长度不匹配

```
ip6_hdr->payload_len = htons(50); // 声称50字节，实际只有10字节  
// 应该被拒绝
```

5. 实验结果

创建 `build` 文件夹：

```
mkdir build  
cd build
```

使用 CMake 构建项目：

```
cmake --build . --target ip6_test
```

运行测试：

```
ip6_test ../testing/data/ip6_test
```

运行完整测试套件后的结果：

Starting IPv6 dual stack unit tests

=====

=== Test IPv6 address conversion functions ===

[PASS]: IPv4 to IPv6 mapped address conversion should succeed

[PASS]: IPv4-mapped IPv6 address format is correct

[PASS]: Should detect IPv4-mapped address

[PASS]: Extracting IPv4 address from IPv6 mapped address should succeed

[PASS]: Extracted IPv4 address should match original

[PASS]: Pure IPv6 address should not be detected as IPv4-mapped

[PASS]: Extracting IPv4 address from pure IPv6 address should fail

=== Test IPv6 packet header parsing ===

[PASS]: IPv6 version field parsed correctly

[PASS]: Traffic class parsed correctly

[PASS]: Flow label parsed correctly

[PASS]: Payload length parsed correctly

[PASS]: Next header field parsed correctly

[PASS]: Hop limit parsed correctly

=== Test IPv6 packet output ===

[PASS]: IPv6 header added correctly

[PASS]: IPv6 version set correctly

[PASS]: Payload length set correctly

[PASS]: Next header field set correctly

[PASS]: Hop limit set correctly

[PASS]: Source IPv6 address set correctly

[PASS]: Destination IPv6 address set correctly

=== Test IPv6 and IPv4-mapped address interoperability ===

[PASS]: IPv6 header added to IPv4-mapped address packet

[PASS]: Destination address set to IPv4-mapped IPv6 address

[PASS]: Packet destination address correctly detected as IPv4-mapped

=== Test IPv6 packet input ===

[PASS]: Packet length is sufficient for IPv6 header

[PASS]: IPv6 version validated

[PASS]: Payload length validated

[PASS]: Destination address is local IPv6 address


```
=== Test IPv6 large packet handling ===
[PASS]: Large packet handled correctly, does not exceed Ethernet MTU
[PASS]: IPv6 header remains correct after truncation

=== Test IPv6 error handling ===
[PASS]: Packet too small is correctly rejected
[PASS]: Packet with wrong version is correctly rejected
[PASS]: Packet with mismatched payload length is correctly rejected

=== Test IPv6 dual stack integration ===
[PASS]: IPv4 address is configured
[PASS]: IPv6 address is configured
Local IPv4 address: 192.168.163.103
Local IPv6 address: 2001:0db8:0000:0000:0000:0000:0000:0001
Local IPv4-mapped IPv6 address: 0000:0000:0000:0000:0000:ffff:c0a8:a367
[PASS]: Local IPv4 address can be correctly mapped to IPv6

=====
Test complete!
Passed: 35 tests
Failed: 0 tests
[OK] All tests passed! IPv6 dual stack implementation is correct.
```

6. 遇到的问题和解决方案

6.1 字节序问题

问题描述：IPv6头部中的多字节字段需要正确处理网络字节序和主机字节序的转换。

具体表现：

- `version_tc_flowlabel` 字段的位操作错误
- `payload_len` 字段的字节序转换遗漏

解决方案：

```
// 正确的版本号设置
uint32_t version_tc_flowlabel = IP_VERSION_6 << 28;
ip6_hdr->version_tc_flowlabel = htonl(version_tc_flowlabel);

// 正确的载荷长度设置
ip6_hdr->payload_len = htons(payload_len);

// 正确的字段解析
uint8_t version = (ntohl(ip6_hdr->version_tc_flowlabel) >> 28) & 0xF;
uint16_t payload_len = ntohs(ip6_hdr->payload_len);
```

6.2 地址映射精度问题

问题描述：IPv4映射IPv6地址的格式检查不够严格，可能误判其他地址为映射地址。

具体表现：

- 只检查了前缀，没有完整验证映射格式
- 边界条件处理不当

解决方案：

```
int is_ip4_mapped_ip6(const uint8_t *ip6_addr) {
    if (!ip6_addr) {
        return 0; // 空指针检查
    }

    // 精确检查前10个字节是否为0
    for (int i = 0; i < 10; i++) {
        if (ip6_addr[i] != 0) {
            return 0;
        }
    }

    // 精确检查第11、12字节是否为0xFF
    if (ip6_addr[10] != 0xFF || ip6_addr[11] != 0xFF) {
        return 0;
    }

    return 1;
}
```

6.3 数据包长度处理问题

问题描述：IPv6数据包的长度计算和验证逻辑不正确。

具体表现：

- 载荷长度计算错误（包含或不包含头部）
- MTU检查逻辑错误

解决方案：

```

// IPv6载荷长度不包括IPv6头部
ip6_hdr->payload_len = htons(buf->len - sizeof(ip6_hdr_t));

// 正确的长度验证
uint16_t payload_len = ntohs(ip6_hdr->payload_len);
if (payload_len + sizeof(ip6_hdr_t) > buf->len) {
    return; // 长度不匹配
}

// MTU检查
int max_payload_len = ETHERNET_MAX_TRANSPORT_UNIT - sizeof(ip6_hdr_t);
if (buf->len > max_payload_len) {
    buf->len = max_payload_len; // 截断处理
}

```

6.4 目标地址检查逻辑问题

问题描述：IPv6数据包接收时的目标地址检查逻辑复杂，容易出错。

具体表现：

- 纯IPv6地址和IPv4映射地址的判断逻辑混乱
- 本机地址匹配逻辑不清晰

解决方案：

```

// 清晰的目标地址检查逻辑
if (memcmp(ip6_hdr->dst_ip, net_if_ip6, NET_IP6_LEN) != 0) {
    // 不是本机IPv6地址，检查是否是IPv4映射地址
    uint8_t ipv4_addr[NET_IP_LEN];
    if (is_ip4_mapped_ip6(ip6_hdr->dst_ip) &&
        ip6_to_ip4_addr(ip6_hdr->dst_ip, ipv4_addr) &&
        memcmp(ipv4_addr, net_if_ip, NET_IP_LEN) == 0) {
        // 是发给本机IPv4的映射地址，继续处理
    } else {
        return; // 不是发给本机的数据包
    }
}

```